

Compiler Register Allocation

01
10

DA Programming Project II

Group T9G2 / Spring 2026

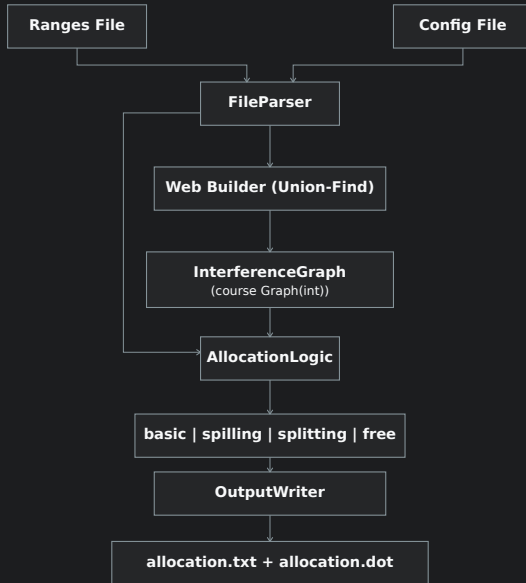


Alexandre Dinis Alves Teixeira
Carlos Francisco de Sousa Ferreira Magalhaes
Diogo
Rodrigo Martins Dias

`make test`

`make docs`

`./register_alloc -b ...`



Spilling and Splitting

01
10

Bounded spilling

Colored Register Allocation
2 register(s), 1 memory web(s)

spills: web0

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
M
1+,2,3-

web1
b
r1
1+,2,3-

web2
c
r0
1+,2,3-

Bounded splitting

Colored Register Allocation
2 register(s), 0 memory web(s)

splits: web0 -> web0,web3

Legend

orange border = split-derived

M = memory

same fill = same register

web0
a
r0
1+,2

web1
b
r1
1+,2,3,4-

web2
c
r0
3+,4,5,6-

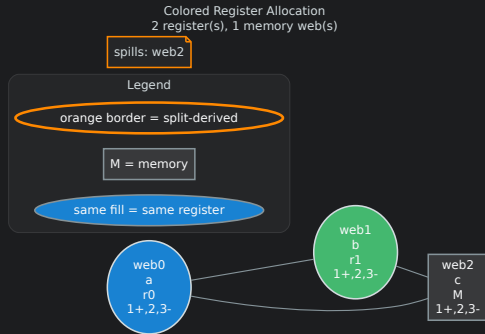
web3
a
r1
5,6

Splitting preserves original markers. 1+,2,5,6- splits into 1+,2 and 5,6- — no fabricated 2- or 5+.
Metadata: spills: N / splits: N.

Custom Free Algorithm

Hybrid strategy

- Edgeless graphs: 1 register
- Complete graphs: K colored, rest spilled
- Bipartite graphs: BFS 2-coloring
- General: DSatur greedy coloring
- Small/medium: branch-and-bound refines spills



Invariant: interfering colored webs never share a register. Memory only for uncolored webs.

Demo and Testing

Inputs for the demo

- `complex_allocation.txt` + `free4.txt`: dense allocator stress test
- `splitting_showcase.txt` + `splitting2.txt`: split reduces K from 3 to 2
- `spilling5.txt`: bounded spilling in a dense graph

```
./register_alloc -b  
  inputs/advanced/ranges/  
complex_allocation.txt  
  inputs/advanced/registers/free4.txt  
  alloc.txt
```

`make test` — 76 unit + 9 integration (passes clean). DOT graphs written alongside allocation output.

Complexities

Input & graph building

Parse: $O(LP)$

**Build
webs:** $O(VR^2P \log P)$

Interference: $O(W^2P + EW)$

Coloring algorithms

Basic: $O(W(W^2 + E))$

Spilling: $O((S + 1)W(W^2 + E))$

Splitting: $O(SQ(W^2P + EW + W(W^2 + E)))$

Free: $O(W^2 + E \log W)$ (heuristic)

Variables: W =webs, E =edges, P =points, R =ranges/var, S =recovery bound, Q =split candidates, L =live ranges.